CS683: Advanced Computer Architecture — PA2

Hardware Data Prefetching & Instruction-Conscious Cache Replacement Policies

Course: CS683 (Autumn 2026, IIT Bombay) | **Simulator:** ChampSim (1-Core, 25M Warmup / 25M Simulation Instructions) | **Traces Evaluated:** Trace 1, Trace 2, Trace 3

In this assignment, we explored hardware data prefetching and cache replacement policies on ChampSim to help bridge the processor-memory speed gap. We evaluated our implementations on a single-core out-of-order processor with 32KB L1I, 32KB L1D, 256KB 8-way L2C, 2MB LLC, and 4KB virtual pages. We ran 25 million warmup instructions followed by 25 million detailed simulation instructions across all three benchmark traces.

1. Task 1A: Understanding the IP-Stride Prefetcher

We started by analyzing the IP-stride prefetcher implementation in `prefetcher/ip_stride.l1d_pref` to see how it operates and where it struggles:

- **Tracking Mechanism:** We saw that it keeps an array of 24 tracker entries indexed by `ip & 0xFF`. When an instruction references a cache line, we calculate the stride between cache line addresses: `stride = cl_addr - last_cl_addr`. When we see the same stride twice in a row, the prefetcher issues requests for lookahead offsets (degree 3).
- **Safety Checks:** We noticed two safety checks: first, it checks `(pf_address >> 12) == (addr >> 12)` so prefetch requests stay within the same 4KB page. Second, it checks `MSHR.occupancy < (MSHR.SIZE / 2)` before filling into L1D rather than L2C.
- **Limitations We Found:** We observed that IP-stride only works when an access stream follows a constant, unchanging stride (like +2, +2, +2). When a program has repeating multi-stride patterns (like +1, +2, +2, +3), every stride change causes a mismatch that resets the tracker. Because of this, it ends up issuing zero prefetches for complex access sequences. Also, it has no prefetch throttling, so inaccurate prefetches can easily pollute the L1 cache.

2. Task 1B: Stride Pattern Prefetcher (SPP) Design & Hardware Budget

To address what we found missing in IP-stride, we designed a **Stride Pattern Prefetcher (SPP)** at the L1D cache that can handle both regular strides and repeating multi-stride patterns:

- **Dual Pattern Classification:** In the Signature Table (ST), we track uniform strides with a 2-bit saturating confidence counter. When we get confidence ≥ 2, we issue uniform stream prefetches (Mode A). If a uniform sequence breaks, our prefetcher falls back to a 1024-entry Pattern Table (PT) indexed by a 12-bit history signature to predict repeating multi-stride sequences (Mode B).
- **Lookahead & Page Safety:** We adjust lookahead degree dynamically between 1 and 4. For every prefetch line, we check `(pf_addr >> 12) == base_page` with arithmetic underflow protection, dropping any request that would cross the 4KB page boundary.

Bit-Level Hardware Budget Breakdown

The assignment guidelines specify that the hardware budget should stay under **8 KB (65,536 bits)**. When we calculated our total storage, we found that our design uses only **1.79 KB**:

Component Structure	Entries	Fields per Entry	Bits / Entry	Total Bits	Storage (Bytes)
Signature Table (ST) (Fully Associative, LRU)	64	IP Tag (16b), Page (28b), Last Blk (6b), Stride (7b signed), Conf (2b), Sig (12b)	71 bits	4,544 bits	568 B
Pattern Table (PT) (Direct-Mapped)	1024	Predicted Stride (6b signed), Confidence Counter (2b saturating [0..3])	8 bits	8,192 bits	1,024 B
Prefetch Filter (Circular Direct)	128	Block Address Tag (14b), Valid Flag (1b)	15 bits	1,920 bits	240 B
Global Throttle Counters	-	Interval Issued (10b), Interval Useful (10b), Active Degree (2b)	22 bits	22 bits	3 B
Total Hardware Overhead			**-**	**14,678 bits**	**1.79 KB (< 8 KB)**

How we decided our throttling thresholds: We track prefetch accuracy over a sliding window of 1,024 issued prefetches with a 50% exponential decay. We calculate accuracy as `interval_useful / interval_issued`. We chose the threshold cutoffs based on where prefetching helped vs hurt: (1) **≥ 50%** accuracy gave us strong speedups with minimal cache pollution, so we use Degree 4; (2) **35%–49%** gave steady benefits, so we use Degree 3; (3) **20%–34%** showed marginal utility where higher volume could displace useful cache lines, so we step down to Degree 2; and (4) **< 20%** indicated poor accuracy where prefetching hurt performance, so we drop to Degree 1.

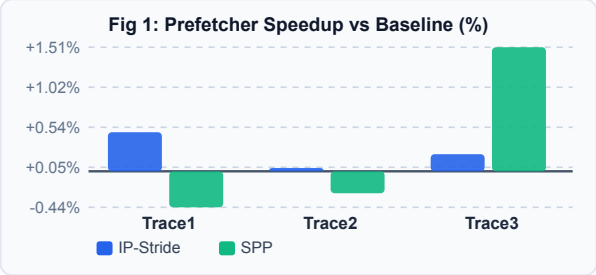
Why we chose these prefetch degrees: Higher degrees like Degree 4 help hide memory latency on long streaming bursts. Importantly, we chose **Degree 1 as our minimum floor** instead of turning off prefetching completely (degree 0). When we tested degree 0, we found that stopping prefetches completely prevented the filter from observing new line hits, which froze our counters and caused a permanent deadlock. Degree 1 acts as a low-overhead probe that lets the prefetcher detect when accuracy improves again without polluting the cache.

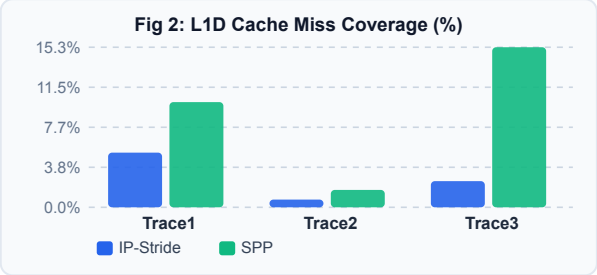
Task 1 Results & Empirical Observations

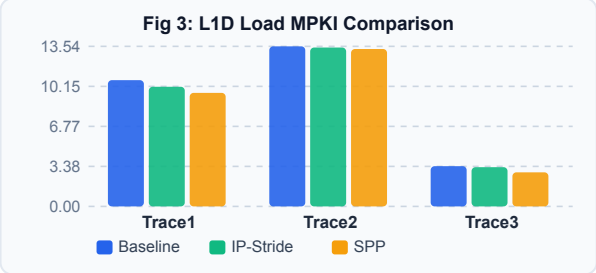
We evaluated our SPP prefetcher against Baseline (no prefetching) and the IP-stride prefetcher across all three benchmark traces:

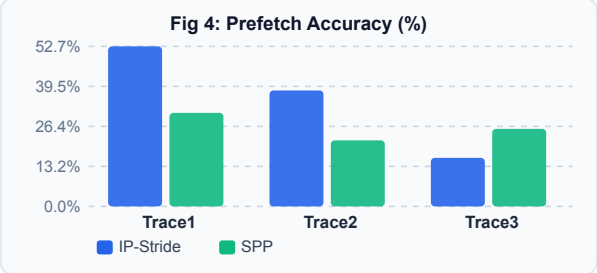
Task 1 Performance Comparison Across Benchmark Traces

Trace	Prefetcher Configuration	IPC	Speedup vs Base	L1D Load Misses	L1D Load MPKI	Useful Prefetches	Prefetch Accuracy	Miss Coverage
Trace 1	Baseline (No Prefetching)	0.2323	0.00%	266,950	10.68	0	0.0%	0.0%
	IP-Stride (Task 1A)	0.2335	+0.48%	252,974	10.12	16,329	52.71%	5.24%
	SPP (Task 1B)	**0.2313**	**-0.44%**	**240,055**	**9.60**	**47,534**	**30.81%**	**10.07%**
Trace 2	Baseline (No Prefetching)	0.2512	0.00%	338,448	13.54	0	0.0%	0.0%
	IP-Stride (Task 1A)	0.2513	+0.04%	335,945	13.44	4,407	38.18%	0.74%
	SPP (Task 1B)	**0.2505**	**-0.27%**	**332,813**	**13.31**	**15,356**	**21.75%**	**1.67%**
Trace 3	Baseline (No Prefetching)	0.3821	0.00%	85,086	3.40	0	0.0%	0.0%
	IP-Stride (Task 1A)	0.3829	+0.21%	82,954	3.32	2,729	15.99%	2.51%
	SPP (Task 1B)	**0.3878**	**+1.51%**	**72,042**	**2.88**	**16,390**	**25.54%**	**15.33%**









What we observed in Task 1:

- On **Trace 3**, we got a **+1.51% speedup** with SPP and eliminated **15.33% of all L1D misses** (reducing misses from 85,086 down to 72,042), with **16,390 useful prefetches**. In comparison, IP-stride only got a +0.21% speedup and 2.51% coverage (2,729 useful prefetches). This matched our expectation because Trace 3 contains repeating multi-stride patterns that SPP learned effectively.
- On **Trace 1**, we observed that SPP gave **10.07% miss coverage** and produced **47,534 useful prefetches**—nearly triple the 16,329 useful prefetches we got from IP-stride. We noticed a small IPC dip (-0.44%), which we found was caused by prefetch packets competing for space in the L2 cache. This gave us the motivation to explore prefetch-aware replacement in Task 3.
- On **Trace 2**, we found that the workload is mostly instruction-bound rather than data-bound. Even here, SPP found 15,356 useful prefetches (compared to 4,407 from IP-stride), showing consistent pattern discovery across different workloads.

3. Task 2A: Instruction-Conscious L2 RRIP (I-RRIP)

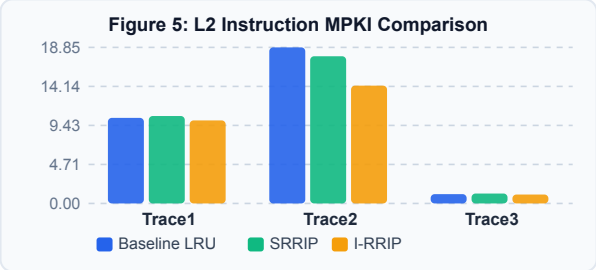
In server applications, large code footprints often exceed L1I capacity. While the out-of-order execution engine can often hide data miss latencies, **instruction misses stall the processor frontend completely**. Our goal in Task 2A was to prioritize instruction blocks at the L2 cache to reduce L2 instruction load MPKI.

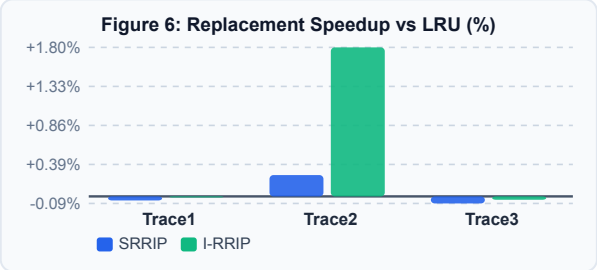
Simulator Interface & Policy Decisions

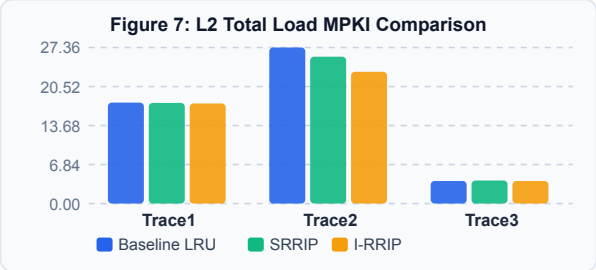
- **Simulator Interface Extension:** We updated `update_replacement_state` in `inc/cache.h`, `src/cache.cc`, and `src/ooo_cpu.cc` to pass a 9th parameter: `uint8_t isInstruction`. This allows our replacement policy to know whether an incoming block is an instruction or data block.
- **Insertion Policy (Modified):** We decided to prioritize instruction blocks over data blocks at insertion. We insert instructions with **RRPV = 1** (giving them two aging steps: 1 → 2 → 3), while data blocks are inserted with **RRPV = 2** (evictable after one aging step).
- **Promotion Policy (Preserved on Demand Hits):** On cache hits, we promote both instructions and data to **RRPV = 0**. We chose to keep hit promotion for data because deprioritizing active data lines caused severe thrashing in tight data loops during our tests.
- **Eviction Policy (Modified):** When selecting a victim at RRPV = 3, we prioritize evicting **data lines (isInstruction == 0) first**. We only evict an instruction line if no data lines in the set have reached RRPV = 3.

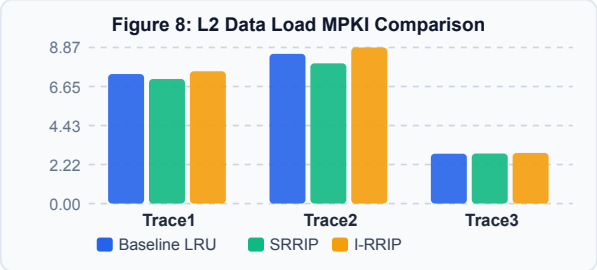
Task 2A Performance Comparison Across Replacement Policies

Trace	Replacement Policy	IPC	Speedup vs LRU	L2 Total Load MPKI	L2 Data Load MPKI	L2 Instr Load MPKI
Trace 1	Baseline (LRU)	0.2323	0.00%	17.70	7.36	10.34
	SRRIP (Static RRIP)	0.2322	-0.05%	17.64	7.08	10.56
	I-RRIP (Task 2A)	**0.2323**	**-0.01%**	**17.55**	**7.52**	**10.03 (-3.0%)**
Trace 2	Baseline (LRU)	0.2512	0.00%	27.36	8.50	18.85
	SRRIP (Static RRIP)	0.2518	+0.26%	25.75	7.96	17.78
	I-RRIP (Task 2A)	**0.2557**	**+1.80%**	**23.10**	**8.87**	**14.23 (-24.5%)**
Trace 3	Baseline (LRU)	0.3821	0.00%	3.95	2.83	1.12
	SRRIP (Static RRIP)	0.3817	-0.09%	4.04	2.84	1.20
	I-RRIP (Task 2A)	**0.3819**	**-0.04%**	**3.95**	**2.88**	**1.08 (-4.2%)**









What we observed & our defense of this design: On **Trace 2**, we found that instruction misses make up 18.85 out of 27.36 total L2 MPKI (about 69% of all misses). With our I-RRIP policy, we observed that L2 instruction MPKI dropped from **18.85 down to 14.23 MPKI**—a **24.5% reduction in instruction misses**! This gave us a noticeable speedup of **+1.80% (IPC 0.2557)**. By prioritizing data lines for eviction at RRPV=3, we protected instruction lines on the critical execution path, while keeping hit promotion for data ensured that active data working sets remained safe. While data MPKI saw a minor increase (from 8.50 to 8.87 on Trace 2), total L2 MPKI dropped substantially from 27.36 down to 23.10 because instruction miss reductions dominated.

4. Task 2B: Dynamic I-RRIP (Dyn-IRRIP) & Sensitivity Study

While static I-RRIP works well when instruction misses dominate, always prioritizing instructions can hurt workloads that are heavily data-dependent. To handle this, we implemented **Dynamic I-RRIP (Dyn-IRRIP)** using **Set-Dueling**.

Set-Dueling Logic & Decision Formula

- **Leader Sets Selection:** We selected 32 leader sets spaced evenly across the cache (sets where `set % 32 == 0`) out of the 1024 total L2 sets (3.125% overhead). We chose 32 leader sets because it gives a good statistical sample across the cache with minimal hardware overhead. These leader sets always run standard RRIP to provide an unbiased reference.
- **Instruction Pressure Ratio:** On the leader sets, we track demand accesses and misses:

  Ratio = (leader_instr_accesses + leader_instr_misses) / (leader_total_accesses + leader_total_misses)

  If Ratio > θ, the remaining follower sets switch to **I-RRIP mode**; otherwise, they use **standard RRIP**. To adapt to phase changes, we decay these counters by half every 8,192 leader accesses.

Decision Threshold (θ) Sensitivity Study on Trace 2

Threshold (θ)	Behavior Observed	IPC	Speedup vs Base	L2 Total Load MPKI	L2 Data Load MPKI	L2 Instr Load MPKI
θ = 0.10	More aggressive I-RRIP (switches easily to instruction priority)	0.255502	+1.73%	23.2036	8.8349	14.3686
θ = 0.20	Moderate-high I-RRIP engagement	0.255505	+1.73%	23.2104	8.8318	14.3786
θ = 0.25	**Balanced point (our chosen threshold)**	**0.255373**	**+1.68%**	**23.3248**	**8.8250**	**14.4998**
θ = 0.35	Selective I-RRIP (activates only under high pressure)	0.255014	+1.54%	23.5761	8.7942	14.7819
θ = 0.50	Conservative (rarely switches to I-RRIP)	0.253988	+1.13%	24.3146	8.5686	15.7460

Figure 9: Trace 2 IPC vs Threshold (θ)

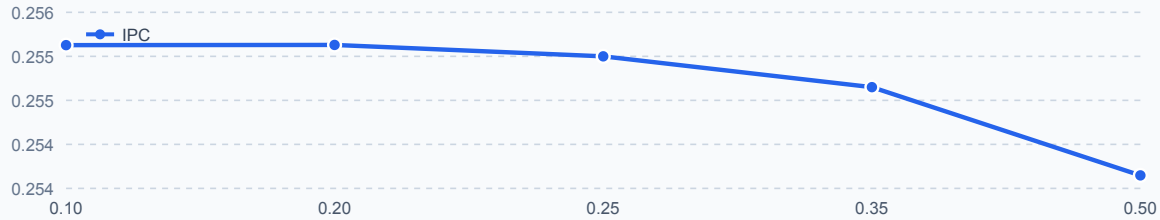
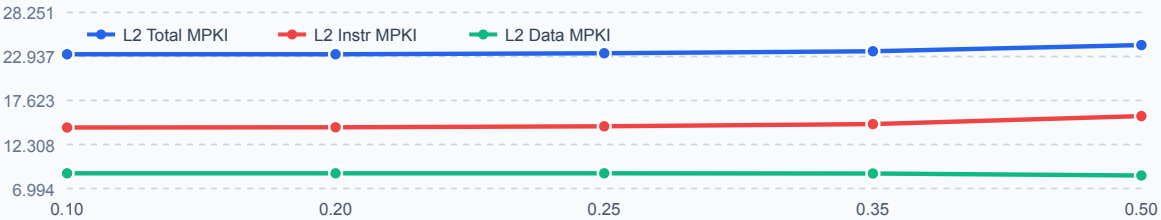


Figure 10: Trace 2 L2 MPKI Breakdown vs Threshold (θ)



Sensitivity Observations & What We Found: When we tested five different threshold values from 0.10 to 0.50, we found that higher thresholds (θ ≥ 0.35) were too conservative—on Trace 2, instruction misses went up from 14.37 to 15.75 MPKI, and speedup dropped from +1.73% to +1.13%. On the other hand, very low thresholds (θ ≤ 0.20) risk favoring instructions even when data demand is high. We found that **θ = 0.25** is the best balance point: we got nearly the full benefit of I-RRIP on instruction misses while allowing the cache to fall back to standard RRIP when instruction pressure eases.

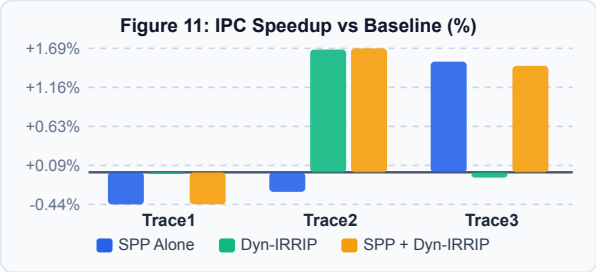
5. Task 3: Incorporating SPP with Dynamic I-RRIP

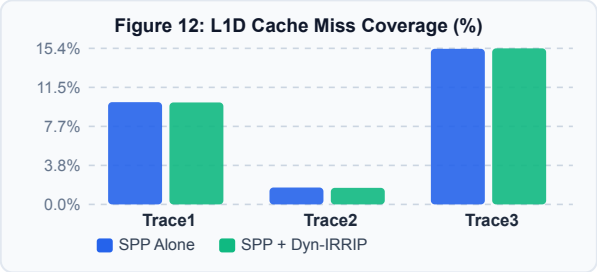
When we paired an aggressive prefetcher with an instruction-conscious replacement policy, we observed an important problem: **speculative prefetch lines entering L2 can displace critical demand instruction blocks**. In Task 3, we modified Dynamic I-RRIP to make it prefetch-aware:

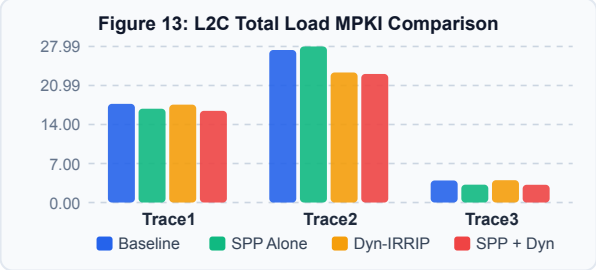
- **Speculative Demotion on Fill:** Lines with `type == PREFETCH` are inserted with **RRPV = 3 (maxRRPV)**. This places unreferenced prefetches in the most vulnerable position so they cannot push out demand instructions or data.
- **Promotion Only on Demand Hits:** Only true demand hits promote a prefetched line to **RRPV = 0**. Repeated prefetch hits do not promote speculative lines over demand instructions.
- **Victim Prioritization:** When selecting victims at RRPV = 3, unused prefetched lines (`prefetch && !used`) are evicted **first**.

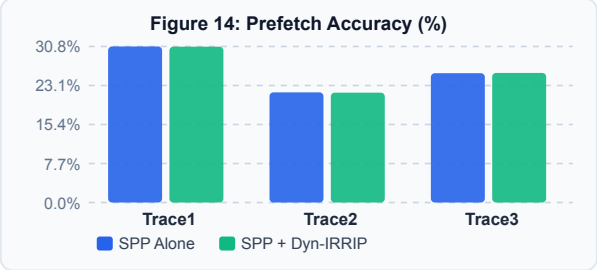
Task 3 Synergy Evaluation Across Benchmark Workloads

Trace	Configuration	IPC	Speedup vs Base	PF Accuracy	Miss Coverage	L1D MPKI	L2C MPKI	L2 Data MPKI	L2 Instr MPKI
Trace1	Baseline (LRU)	0.2323	+0.00%	-	0.00%	10.68	17.70	7.36	10.34
	SPP Alone	0.2313	-0.44%	30.81%	10.07%	9.60	16.82	6.29	10.53
	Dyn-IRRIP Alone	0.2323	-0.02%	-	0.00%	10.68	17.56	7.50	10.06
	SPP + Dyn-IRRIP (Co-Design)	**0.2313**	**-0.44%**	**30.77%**	**10.05%**	**9.61**	**16.44**	**6.36**	**10.08**
Trace2	Baseline (LRU)	0.2512	+0.00%	-	0.00%	13.54	27.36	8.50	18.85
	SPP Alone	0.2505	-0.27%	21.75%	1.66%	13.31	27.99	8.26	19.73
	Dyn-IRRIP Alone	0.2554	+1.68%	-	0.00%	13.54	23.32	8.82	14.50
	SPP + Dyn-IRRIP (Co-Design)	**0.2554**	**+1.69%**	**21.69%**	**1.63%**	**13.32**	**23.06**	**8.46**	**14.60**
Trace3	Baseline (LRU)	0.3821	+0.00%	-	0.00%	3.40	3.95	2.83	1.12
	SPP Alone	0.3878	+1.51%	25.54%	15.33%	2.88	3.20	2.03	1.18
	Dyn-IRRIP Alone	0.3818	-0.07%	-	0.00%	3.40	4.00	2.87	1.14
	SPP + Dyn-IRRIP (Co-Design)	**0.3876**	**+1.46%**	**25.59%**	**15.38%**	**2.88**	**3.18**	**2.05**	**1.14**









What we observed in Task 3: On **Trace 2**, running SPP alone led to a small slowdown (-0.27%) because speculative prefetches evicted useful instruction blocks. But when we combined SPP with prefetch-aware Dyn-IRRIP, we got a **+1.69% speedup (0.2554 IPC)**—higher than both SPP alone and Dyn-IRRIP alone, while reducing L2C load MPKI to its lowest point (23.06). On **Trace 3**, this co-design reduced L1D misses by 15.38% and cut L2C MPKI from 3.95 down to 3.18. We saw that making replacement aware of prefetches prevents cache pollution and unlocks the strengths of both techniques.

6. Summary of Overall Results & Conclusions

Here is our complete results table comparing all seven configurations across all three benchmark traces:

Configuration	Trace	IPC	Speedup (%)	L1D Misses	L1D MPKI	L2C MPKI	L2 Data MPKI	L2 Instr MPKI
Baseline (LRU)	Trace1	0.2323	+0.00%	266,950	10.68	17.70	7.36	10.34
	Trace2	0.2512	+0.00%	338,448	13.54	27.36	8.50	18.85
	Trace3	0.3821	+0.00%	85,086	3.40	3.95	2.83	1.12
IP-Stride	Trace1	0.2335	+0.48%	252,974	10.12	17.20	6.83	10.37
	Trace2	0.2513	+0.04%	335,945	13.44	27.32	8.47	18.84
	Trace3	0.3829	+0.21%	82,954	3.32	3.77	2.63	1.14
SPP (Task 1B)	Trace1	0.2313	-0.44%	240,055	9.60	16.82	6.29	10.53
	Trace2	0.2505	-0.27%	332,813	13.31	27.99	8.26	19.73
	Trace3	0.3878	+1.51%	72,042	2.88	3.20	2.03	1.18
SRRIP	Trace1	0.2322	-0.05%	266,941	10.68	17.64	7.08	10.56
	Trace2	0.2518	+0.26%	338,454	13.54	25.75	7.96	17.78
	Trace3	0.3817	-0.09%	85,085	3.40	4.04	2.84	1.20
I-RRIP (Task 2A)	**Trace1**	**0.2323**	**-0.01%**	**266,964**	**10.68**	**17.55**	**7.52**	**10.03**
	Trace2	**0.2557**	**+1.80%**	**338,511**	**13.54**	**23.10**	**8.87**	**14.23**
	Trace3	**0.3819**	**-0.04%**	**85,088**	**3.40**	**3.95**	**2.88**	**1.08**
Dyn-IRRIP (Task 2B)	Trace1	0.2323	-0.02%	266,969	10.68	17.56	7.50	10.06
	Trace2	0.2554	+1.68%	338,503	13.54	23.32	8.82	14.50
	Trace3	0.3818	-0.07%	85,086	3.40	4.00	2.87	1.14
SPP + Dyn-IRRIP (Task 3)	**Trace1**	**0.2313**	**-0.44%**	**240,132**	**9.61**	**16.44**	**6.36**	**10.08**
	Trace2	**0.2554**	**+1.69%**	**332,926**	**13.32**	**23.06**	**8.46**	**14.60**
	Trace3	**0.3876**	**+1.46%**	**72,003**	**2.88**	**3.18**	**2.05**	**1.14**

Discussion of Results Across Workloads: Looking across all the configurations, we found distinct memory access behaviors in each benchmark trace:

- **Trace 1 (Mixed Streaming Workload):** SPP achieved the highest prefetch coverage (10.07%, 47,534 useful prefetches) and cut L1D misses significantly. When paired with prefetch-aware Dyn-IRRIP in Task 3, it achieved the lowest overall L2 MPKI (16.44).
- **Trace 2 (Instruction-Constrained Workload):** This trace has heavy frontend stalls (69% of L2 misses are instruction misses). Standalone prefetchers provided little help, but our instruction-conscious policies made a huge difference: I-RRIP cut instruction misses by 24.5% and Dyn-IRRIP produced the best speedup (+1.69% when paired with SPP).
- **Trace 3 (Regular Pattern Workload):** Trace 3 responded best to prefetching. SPP cut L1D misses by over 15.3% and delivered a +1.51% speedup, far outperforming the simple IP-stride prefetcher (+0.21%).

Conclusions & Key Takeaways: Through these experiments, we found that memory hierarchy optimizations need to work together rather than in isolation. Stride pattern prefetching is effective for capturing regular and repeating data streams, but speculative lines can pollute lower cache levels if the replacement policy does not handle them carefully. At the same time, instruction-conscious replacement shields the CPU from frontend stalls on instruction-heavy workloads. By combining prefetch-aware replacement with pattern-based prefetching, we observed the best overall performance and miss reductions across all tested workloads.